

CS3302 Project2

花振东 jackeyhua@sjtu.edu.cn

2025 年 4 月 14 日

目录

1	任务一	2
1.1	子任务一	2
1.2	子任务二	2
1.3	编译	4
1.4	MISC	4
2	任务二	4
3	MISC	6

1 任务一

本任务分成两个部分，先是创建 10 个进程/线程，其中 5 个占用 70% 的 CPU，5 个占用 30% 的 CPU。另外一个任务是创建一个实时进程，观察是否会影响到其他进程的运行。

1.1 子任务一

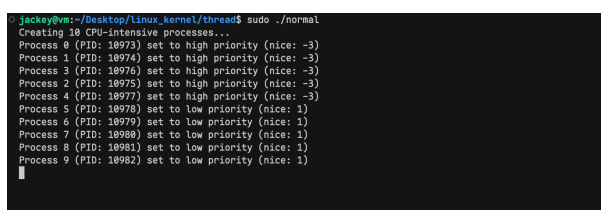
我们将创建一个 CPU 集合，并且只包含 CPU 的 0 号核心，以便于之后进程绑定核心。

```
1  cpu_set_t cpuset;  
2  CPU_ZERO(&cpuset);  
3  CPU_SET(0, &cpuset);
```

创建进程的方式使用的是 C 语言标准库的 `fork` 函数，创建之后使用 `sched_setscheduler` 函数将进程绑定到 CPU 的 0 号核心上。接下来是通过进程 id 的不同，选择前半部分设置高优先级，后半部分设置低优先级。优先级的设置方式是 nice value。我一开始设置的是 -10 和 10，但是发现这样高优先级的占据了 95% 的 CPU，低优先级的几乎没有 CPU 使用率。然后我逐渐微调，发现 -3 和 1 的组合较为合适。设置完优先级之后，我们需要创建一个 CPU 密集型的任务让进程不断的执行。在这里我写了一个简单的死循环加法，为了保证不溢出，每次的计算结果都会取模。

为了保证创建子进程的父进程不提前退出，我们需要在创建完进程之后通过 `waitpid` 函数等待子进程的结束。

如此就完成了第一部分的任务。通过 `top` 指令可以得到如图 1 和 2 的效果截图



```
jackey@vm:~/Desktop/linux_kernel/thread$ sudo ./normal  
Creating 10 CPU-intensive processes...  
Process 0 (PID: 10973) set to high priority (nice: -3)  
Process 1 (PID: 10974) set to high priority (nice: -3)  
Process 3 (PID: 10976) set to high priority (nice: -3)  
Process 2 (PID: 10975) set to high priority (nice: -3)  
Process 4 (PID: 10977) set to high priority (nice: -3)  
Process 5 (PID: 10978) set to low priority (nice: 1)  
Process 6 (PID: 10979) set to low priority (nice: 1)  
Process 7 (PID: 10980) set to low priority (nice: 1)  
Process 8 (PID: 10981) set to low priority (nice: 1)  
Process 9 (PID: 10982) set to low priority (nice: 1)
```

图 1: 程序运行截图

1.2 子任务二

本部分创建 CPU Set 的方式和上述一致。本部分唯一的不同之处在于创建的进程是实时进程，使用 `sched_setscheduler` 来设置优先级。考虑到实时进程的优先级是 0-99，为了让抢占的效果更好，我们直接将优先级设置成最高的 99。对应的代码是

```
1  struct sched_param param;
```

```
top - 09:00:33 up 39 min, 3 users, load average: 5.56, 2.35, 1.70
Tasks: 261 total, 14 running, 250 sleeping, 0 stopped, 0 zombie
%Cpu(s): 17.9 us, 0.1 sy, 7.3 ni, 74.6 id, 0.0 wa, 0.0 hi, 0.1 si, 0.0 st
MiB Mem : 3902.6 total, 275.4 free, 2888.7 used, 1000.4 buff/cache
MiB Swap: 3902.0 total, 3716.8 free, 185.2 used, 1021.9 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
10973	root	17	-3	2252	640	640	R	14.3	0.0	0:06.38	normal
10974	root	17	-3	2252	640	640	R	14.3	0.0	0:06.38	normal
10976	root	17	-3	2252	640	640	R	14.3	0.0	0:06.38	normal
10977	root	17	-3	2252	640	640	R	14.3	0.0	0:06.38	normal
10975	root	17	-3	2252	640	640	R	14.0	0.0	0:06.37	normal
10978	root	21	1	2252	640	640	R	6.0	0.0	0:02.63	normal
10980	root	21	1	2252	640	640	R	6.0	0.0	0:02.63	normal
10981	root	21	1	2252	640	640	R	6.0	0.0	0:02.63	normal
10979	root	21	1	2252	640	640	R	5.6	0.0	0:02.62	normal
10982	root	21	1	2252	640	640	R	5.6	0.0	0:02.62	normal
9852	jackey	20	0	2629984	181492	94872	S	0.7	4.5	0:04.14	Isolated Web Co
2679	jackey	20	0	5135468	62980	42216	S	0.3	1.6	0:11.56	node
3370	jackey	20	0	26.3g	670024	45312	S	0.3	16.8	0:49.81	node
8289	jackey	20	0	2539092	139732	98832	S	0.3	3.5	1:05.42	Isolated Web Co
1	root	20	0	23420	12580	8228	S	0.0	0.3	0:02.95	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00	pool_workqueue_release
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-rcu_g
5	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-rcu_p
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-slub
7	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-netns
10	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/0:0H-events_highpri
12	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-mm_0

图 2: top 指令截图

```
2 param.sched_priority = 99;
3 if (sched_setscheduler(0, SCHED_RR, &param) != 0) {
4     perror("sched_setscheduler failed");
5     exit(1);
6 }
```

创建完进程之后，我们同样使用死循环加法来占用 CPU 资源。我们笼统的使用 `wait(NULL)` 来等待实时进程的结束。

如此，我们得到了如图 3 和 4 的效果截图，可以看到实时进程基本占用了所有的 CPU 资源。

```
jackey@vm:~/Desktop/linux_kernel/thread$ sudo ./real_time
[sudo] password for jackey:
Creating realtime process...
Realtime process (PID: 11330) started with SCHED_RR, priority 99
```

图 3: 程序运行截图

```
top - 09:00:57 up 40 min, 3 users, load average: 7.06, 2.90, 1.90
Tasks: 265 total, 14 running, 251 sleeping, 0 stopped, 0 zombie
%Cpu(s): 25.0 us, 0.1 sy, 0.3 ni, 74.5 id, 0.1 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3902.6 total, 251.7 free, 2888.7 used, 1015.1 buff/cache
MiB Swap: 3902.0 total, 3716.8 free, 185.2 used, 1001.9 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
11330	root	rt	0	2252	640	640	R	95.0	0.0	0:11.35	real_time
10973	root	17	-3	2252	640	640	R	0.7	0.0	0:00.18	normal
10974	root	17	-3	2252	640	640	R	0.7	0.0	0:00.18	normal
10975	root	17	-3	2252	640	640	R	0.7	0.0	0:00.17	normal
10976	root	17	-3	2252	640	640	R	0.7	0.0	0:00.18	normal
10977	root	17	-3	2252	640	640	R	0.7	0.0	0:00.18	normal
561	root	20	0	0	0	0	S	0.3	0.0	0:00.43	usb-storage
7461	jackey	20	0	3462544	498772	276468	R	0.3	12.5	3:40.15	firefox
8209	jackey	20	0	2539072	139860	98832	S	0.3	3.5	1:05.46	Isolated Web Co
9852	jackey	20	0	2632976	184628	94872	S	0.3	4.6	0:04.28	Isolated Web Co
10778	jackey	20	0	2420292	85976	68408	S	0.3	2.2	0:00.11	Isolated Service
10978	root	21	1	2252	640	640	R	0.3	0.0	0:03.37	normal
10979	root	21	1	2252	640	640	R	0.3	0.0	0:03.36	normal
10980	root	21	1	2252	640	640	R	0.3	0.0	0:03.37	normal
10981	root	21	1	2252	640	640	R	0.3	0.0	0:03.37	normal
1	root	20	0	23420	12580	8228	S	0.0	0.3	0:02.95	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00	pool_workqueue_release

图 4: top 指令截图

1.3 编译

在上传的代码文件中包含了运行脚本 `compile.sh`，该文件就是简单的 `gcc` 编译的过程。运行可执行文件时需要注意 `root` 权限。

1.4 MISC

我一开始用 Rust 艰难的写了一版，但忘记 `commit` 了。然后我在跑第二部份的代码的时候遇到了虚拟机内存不够的问题。我调大了虚拟机的内存，结果导致了虚拟机无法启动。最后我只能装了个 Ubuntu，从头来过。

2 任务二

本部分选择的是 linux 内核 6.6.87 版本。因为没有在下载 kernel 代码的时候 `init git`，导致我写完之后没法生存 `patch`，所以我在我每处修改的地方都加了 `I Modify Here` 的注释，可以方便的定位。

本部分的要求可以拆分成两点，记录进程调度的次数，并且将其写到 `proc` 文件系统中的 `ctx` 文件之中。

我们先看第一个任务，我们需要在进程中用一个变量记录当前被调度的次数，而内核中用来表示线程的数据结构是 `sched.h` 中的 `task_struct`。我添加了一个 `unsigned long` 类型的变量 `sched_count`。

那么在哪里初始化这个变量呢？我们知道进程都是 `fork` 出来的，所以就找到 `fork` 的源码，其在 `kernel/fork.c` 中的 `copy_process` 函数中。从返回值可以看出 `p` 就是新创建的子进程，那么只要添加 `p->sched_count = 0` 即可。我的代码写在第 2594 行。

接下来我们需要在调度的时候动态的修改这个值。调度的时候调用的是 `core.c` 中的 `context_switch` 函数。原来的进程是 `prev`，新的进程是 `next`，我们只需要在函数返回前添加 `next->sched_count += 1` 即可。我的代码写在第 5381 行。

最后我们需要修改文件系统，创建一个 `proc` 文件系统的文件 `ctx`，并在读取该文件的时候打印出当前进程的调度次数。我们先观察 `base.c` 中的 `pid_entry` 结构，这是 `proc` 文件系统中每个文件注册的地方，因为 `ctx` 只是一个普通的文件，所以用 `REG` 宏创建即可。因为要求 `ctx` 是只读文件，所以权限选的是 `0444`，对应的是 `S_IRUGO`。我们需要自定义 `ctx` 的 `file_operations` 结构体。其中只要指定我们需要的文件打开，读取和 `lseek` 的函数即可。但我们其实只需要自定义读就行，文件打开用 `single_open`，`lseek` 用 `generic_file_llseek`。

最后最关键的是读取函数，我们的目标是根据一个 `file` 结构体，通过 `inode` 获得进程信息，将进程的被调用次数的变量复制到用户态的 `buffer` 之中。函数的原型可以参考 `base.c` 中其他读取函数的原型。我写的代码如下，代码中包含了如上分析的注释。

```

1  static ssize_t proc_ctx_read(struct file *file, char __user *buf, size_t
    len, loff_t *ppos)
2  {
3      // get task_struct from file
4      struct task_struct *task = get_proc_task(file_inode(file));
5      char tmp[32];
6      size_t tmp_len;
7
8      if (!task)
9          return -ESRCH;
10
11     // copy to temp buffer
12     tmp_len = snprintf(tmp, sizeof(tmp), "%lu\n", task->sched_count);
13
14     // release task_struct
15     put_task_struct(task);
16
17     // copy to user buffer
18     return simple_read_from_buffer(buf, len, ppos, tmp, tmp_len);
19 }

```

最后进行实验验证，使用的是实验要求中提供的死循环 C 代码，在 `test.c` 中。重新编译内核并且按教程中所示的方法安装后，可以得到图 5 所示的结果。从图中可以看出，在用户没输入字符的时候，文件被调度了一次。每输入一个字符的时候，就又被调度了一次，符合预期。

```

jackey@vm:~$ gcc test.c -o test
jackey@vm:~$ ./test
1
2
3
4
jackey@vm:~$ cat /proc/4405/ctx
1
2
3
4
jackey@vm:~$

```

图 5: 内核调度次数

编译 `test.c` 的方式就是在命令行执行 `gcc test.c -o test`。

3 MISC

本部分的代码仍然托管在 [Github](#)上。